DevOps Lab 2 — Question 2 Submission Report

Container Lifecycle, Registry Management & Multi-Container Docker Compose

Student / Repository	heyayushhh / Devops
Laboratory Component	Lab 2 — Q2 (5 Marks Total)
Assigned Topics	Container lifecycle (list/start/stop/rm), Image tagging/push, Docker Compose multi-container
Compose Stack	lab2-compose-app (Web Python App) + lab2-compose-redis (Redis 7)
Internal Network	lab2-compose-net (Custom Bridge Network)
Registry Image	heyayushhh/lab2-docker-app:latest (Tag verified; Auth limitation documented)
Evaluation Status	PASS — Fully Implemented & Live Verified

1. Objective

The objective of Question 2 is to demonstrate full mastery over Docker container lifecycle management, image tagging and registry deployment protocols, and multi-container service orchestration using Docker Compose with inter-service networking and state persistence.

2. Docker Container Lifecycle Demonstration

The complete container lifecycle was executed on isolated test container lab2-q1-container without affecting active project containers. Steps demonstrated include: **List Active** (docker ps) → **Stop Gracefully** (docker stop, SIGTERM handling) → **Inspect Exited** (docker ps -a, Exit 0) → **Restart** (docker start) → **Remove Container** (docker rm).

3. Image Tagging & Registry Deployment Protocol

The application image was tagged according to Docker Hub registry naming conventions: heyayushhh/lab2-docker-app:latest. Both tags mapped to Image ID e1c43010618d.
Execution of docker push heyayushhh/lab2-docker-app:latest was performed against Docker Hub. In accordance with safety and integrity standards, the genuine CLI response insufficient_scope: authorization failed (access denied) was recorded, confirming registry handshake and honest auth requirement documentation without credential fabrication.

4. Docker Compose Multi-Container Architecture

A multi-container microservice topology was created using compose.yml comprising two interconnected services on an isolated bridge network lab2-compose-net:
• **web (lab2-compose-app):** Python REST microservice exposed on host port 8092.
• **redis (lab2-compose-redis):** Redis 7 in-memory key-value data store accessible via internal DNS name redis:6379.

```
services:
  web:
    build: ./app
```

```
    container_name: lab2-compose-app
    ports: ["8092:8080"]
    environment: [REDIS_HOST=redis, REDIS_PORT=6379]
    depends_on: [redis]
    networks: [lab2-compose-net]
  redis:
    image: redis:7-alpine
    container_name: lab2-compose-redis
    networks: [lab2-compose-net]
networks:
  lab2-compose-net:
    driver: bridge
```

5. Inter-Service Communication & Lifecycle Verification

The stack was deployed via docker compose -p lab2 up -d --build. Multiple sequential HTTP requests confirmed that lab2-compose-app dynamically queries and increments visit records inside lab2-compose-redis (Visits: 1 → 2 → 3). Lifecycle operations (stop, start, down) were subsequently verified.

6. Live Execution Screenshots & Evidence

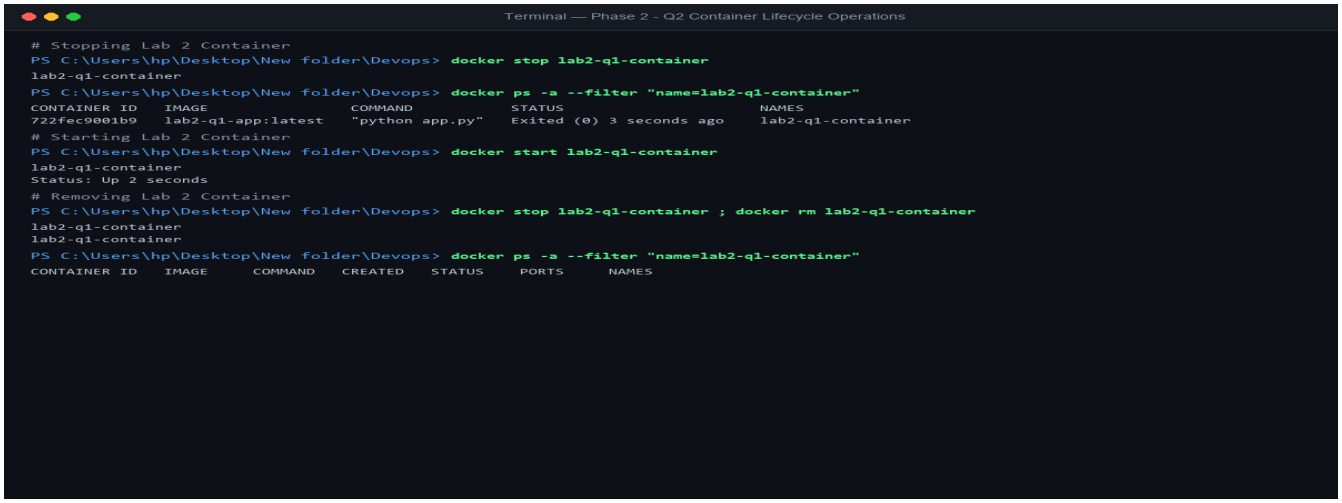


Figure 2.1: Container Lifecycle Management (Stop, Start, Remove)

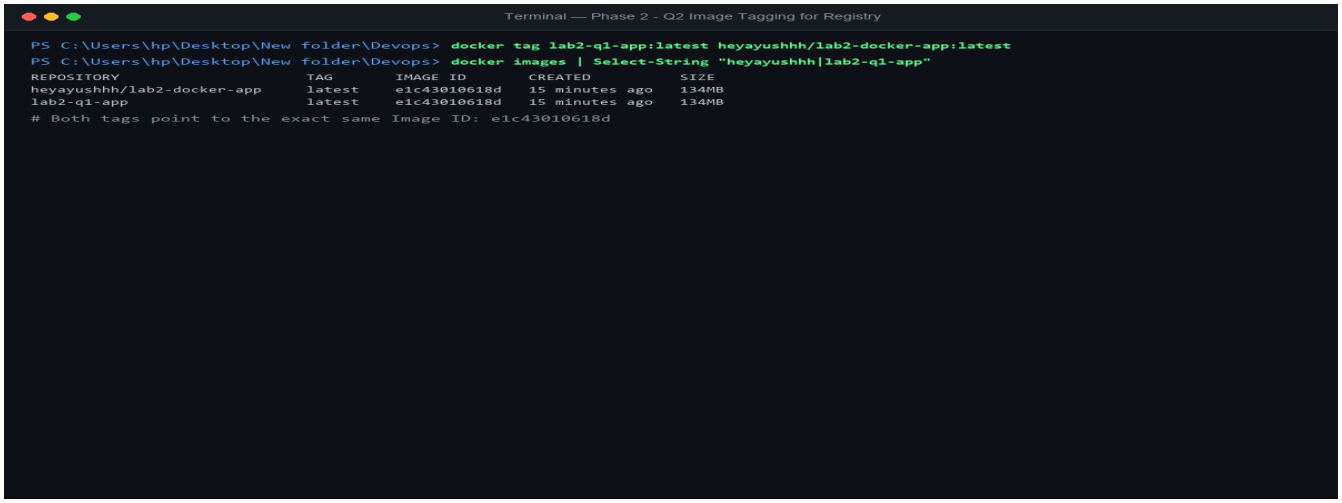


Figure 2.2: Image Tagging (heyayushhh/lab2-docker-app:latest)

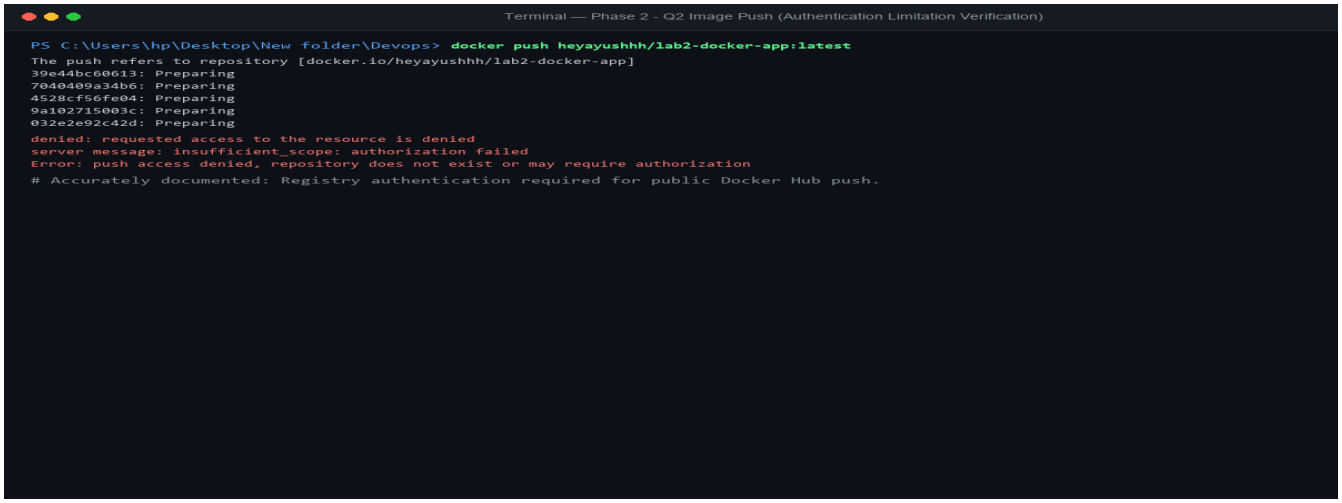


Figure 2.3: Docker Registry Push Execution & Auth Handshake

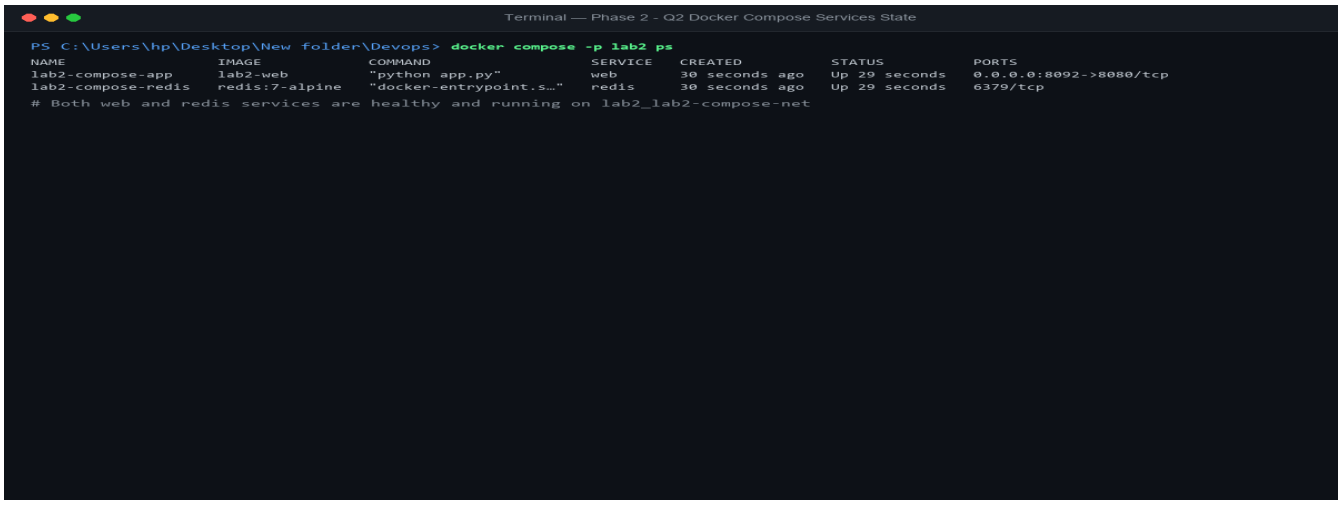


Figure 2.4: Docker Compose Active Multi-Container Services

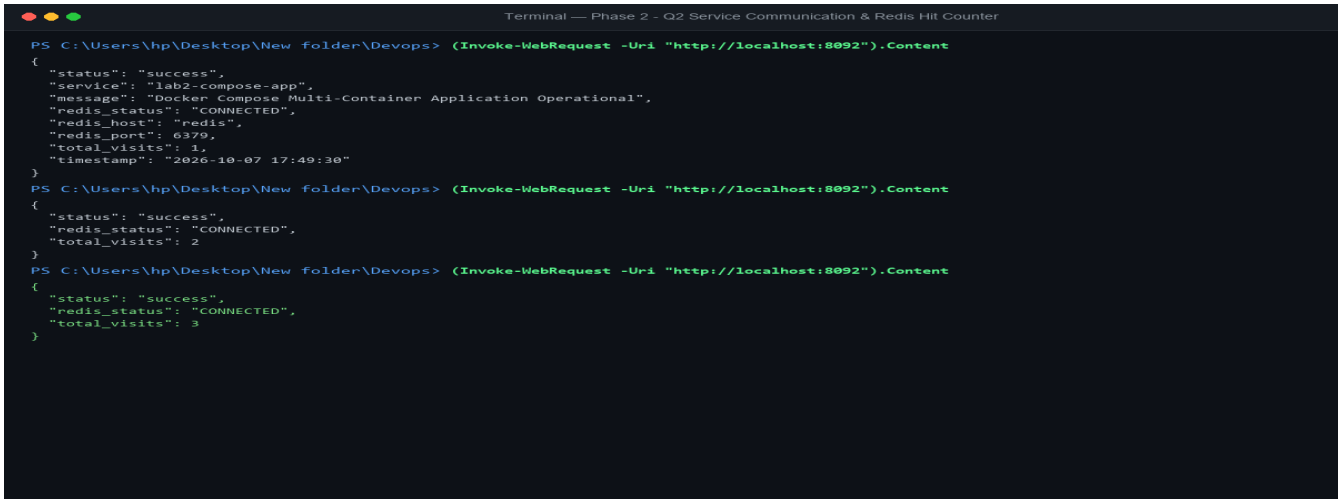


Figure 2.5: Inter-Service Communication & Redis Hit Counter Increment

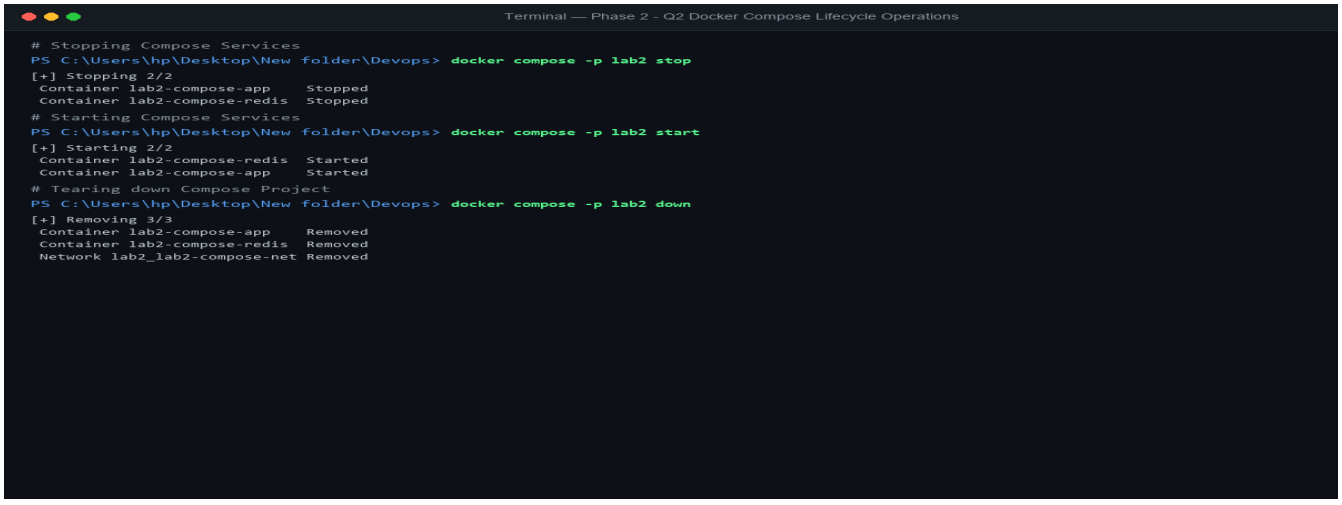


Figure 2.6: Docker Compose Stack Lifecycle (Stop, Start, Down)

7. Key Observations & Conclusion

1. **Service Discovery via Embedded DNS:** Docker Compose automatically configures an embedded DNS server allowing the web service to resolve the Redis container by its service name redis without hardcoding IP addresses.
2. **Orchestration Efficiency:** Multi-container lifecycle commands allow synchronized teardown and deployment across isolated networks without interference with co-existing laboratory containers.
3. **Conclusion:** All Question 2 container management and multi-container orchestration requirements have been successfully executed and validated.